Lab 6: **Servo**

SED 1115 – Embedded Programming and Scripting
Faculty of Engineering – University of Ottawa

Lab Objectives

The purpose of this lab is to learn how to use a servo motor with the Raspberry Pi Pico. You will learn how to control the position of the servo motor using PWM signals.

Lab Instructions

Parts and Equipment

You will need: a Raspberry Pi Pico, a laptop or other computer with USB interfaces and internet access, and a USB micro-B cable that works with that computer.

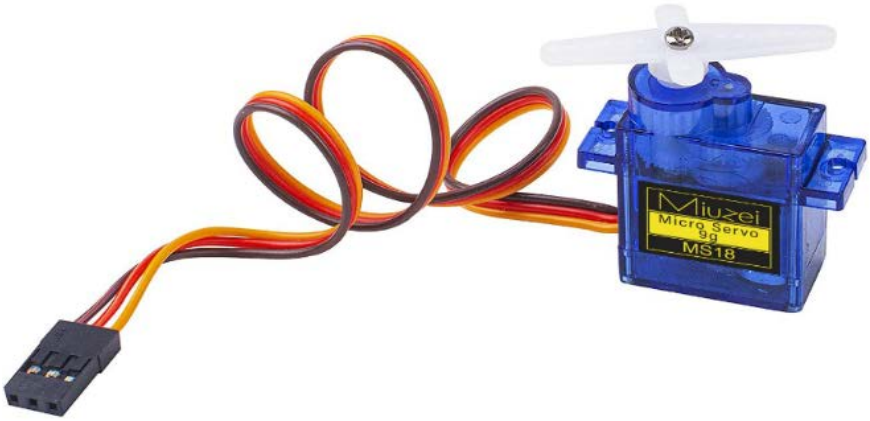
Warnings

It is recommended that you place the Pico (and/or the expansion board) on a flat, non-conductive, anti-static surface, such as a wooden table or a notebook. Electrostatic discharge (ESD) can damage the Pico (or most other electronic devices). To avoid such damage, when working on the Pico, ground yourself using an approved grounded ESD wrist strap (properly connected to ground) or touch a grounded metal part of a grounded object (e.g. your computer case), constantly, while working with any ESD-sensitive semiconductor devices. Try to touch only the metal standoffs of the accessory board.

Servo Motor

A servo motor is a rotary actuator that allows for precise control of angular position. It consists of a motor coupled to a sensor for position feedback. It also requires a servo controller to provide the appropriate control signals. Servo motors are used in many applications, including robotics, radio-controlled vehicles, and industrial automation.

The servo used in this lab is a model SG90 (also known as a "9g" servo). Weighing about 9 grams, it is a small, inexpensive, and widely available servo motor. It is a 3-wire device, with a brown wire for the ground, a red wire for power, and an orange wire for the control signal.



The servos that you are going to use in this lab are designed to run with a supply voltage of 4.8V, but they will work with a voltage of 5V too. The manufacturer claims that the device has a stall torque value of 1.3 kg-cm. This means that it can exert a force of 1.3 kg at a distance of 1 cm, measured from the centre of the shaft (or, equally, it can exert a force of 1kg at a distance of 1.3 cm, etc.). It has a maximum rotation of 180 degrees, plus or minus 3 degrees.

Servo Control Signals

As already stated, the servo motor requires a control signal to determine its position. The control signal is a pulse-width modulated (PWM) signal with a frequency of 50 Hz. A 50Hz waveform has a clock period of 20ms or 20,000 microseconds.

The duty cycle (i.e. the percentage of time that the signal is high during the clock period) of the 50Hz signal determines the position of the servo motor. The servo that you are going to use in this lab accepts pulse widths that vary from 500 microseconds to 2500 microseconds. Pulse widths that are outside this range will not work and may even damage the servo.

A pulse width of 500 microseconds corresponds to a position of 0 degrees and a pulse width of 2500 microseconds corresponds to a position of 180 degrees. The various position values that lie in between these values can be interpolated in a linear fashion. Not all servos have the same range of pulse widths. Some servos may have a range of 1000 to 2000 microseconds, so you should always check the specifications of the servo that you are using.

Servo Control with PWM

As you have learned in the second lab, the Raspberry Pi Pico has 16 different PWM output channels. You can control the output of each channel using the `PWM` class. You have used `PWM` class to control the brightness of an LED, and the same method can be used to control the position of a servo motor.

We can create a `PWM` object to control the servo motor as follows:

```
from machine import Pin, PWM

servo = PWM(Pin(0))
servo.freq(50)
```

Note that this example assumes that the servo motor is connected to GPIO 0. You can use any other GPIO pin that supports PWM. Use https://pico.pinout.xyz/ to find out which GPIO pins support PWM. Before you can control the position of a servo motor, you need to know how to convert the desired position of the servo motor to the proper input value to be sent to the `duty_u16` method of the `PWM` class.

We know that a pulse width of 500 microseconds corresponds to a position of 0 degrees, and we also know that a pulse width of 2500 microseconds corresponds to a position of 180 degrees. Based on this, we can determine the pulse width for any position, using a straight-line interpolation formula. For example, if we want to set the position of the servo motor to X degrees, where 0 < X < 180, we can calculate the pulse width as follows:

```
pulse_width = 500 + (2500 - 500) * X / 180
```

However, the duty_u16 method of the PWM class requires an integer value between 0 and 65535. This value corresponds to the duty cycle of the PWM signal, which is the ratio of the high pulse width to the period of the PWM signal. We know that the frequency of the PWM signal is 50 Hz, which means that the period of the PWM signal is 1 second / 50 Hz = 20000 microseconds. Therefore, the duty cycle of the PWM signal is:

```
duty_cycle = pulse_width / 20000
```

This will give us a value between 0 and 1. We can convert this value to an integer value between 0 and 65535 by multiplying it by 65535. Therefore, the final value we need to pass to the `duty_u16` method of the `PWM` class is:

```
servo.duty_u16(int(duty_cycle * 65535))
```

Questions

1. What is the pulse width (in microseconds) to put the servo in a position of 45 degrees?

_____

2. What is the maximum and minimum value that you can safely pass to the `duty_u16` method, *without* potentially damaging the servo motor?

_____

3. Knowing these values, how can you simplify the calculation of the input value to the `duty_u16` method (i.e what would the combined formula look like)?

_____

_____

_____

Lab Assignment

In this lab, you will write a method called "`translate`" that takes a position in degrees and returns the value to be passed to the `duty_u16` method of the PWM class. You will then use this method, in future labs and for your group project, to control the position of the servo motor.

A template for this method is provided in the file `servo_translator.py`. You should complete this method and test it using the `servo_translator_unittest.py` file. You can run the test using the following command:

```
python servo_translator_unittest.py
```

If all of the tests, defined in the test class, pass, then you should see an "OK" message. Otherwise, you will see a message that indicates which test (or which tests) failed. You should make sure that your code passes all tests, *before* submitting it for grading.

In `servo_translator.py`, do not modify the signature (name, input, return type) of the `translate` function, otherwise, the tests will fail. You can add any helper functions that you need to the file. A helper function performs part of the computation of another function and is used to make code simpler or more understandable, clearer and more modular and so generally easier to re-use/maintain (e.g. defining a '`square`' helper function for use inside an '`calculate_area`' function) .

Note that you also should not change the content of the `servo_translator_unittest.py` file. You don't have to submit this file, since this has been written for you and not written by you. As already stated, please only submit the `servo_translator.py` file.

Lab Submission:

Please submit the filled-in laboratory manual PDF file and your `servo_translator.py` into BrightSpace.

The code should be well-formatted and well-commented. The code should be easy to read and easy to understand. It is more than just OK for you to submit a reference to your github too, although the files still need to be double-submitted into BrightSpace, for marking purposes.